Hands on for Bayesian

Part 1

Experiment Objectives:

Master the principles and learning algorithm of Bayesian.

Experiment Requirements:

Implement the Naive Bayes algorithm by programming it yourself without using sklearn's MultinomialNB, and apply it to the watermelon 3.0 dataset discussed in the course.

Given a new test sample (e.g., ['乌黑', '硬挺', '沉闷', '清晰', '稍凹', '软粘', 0.500, 0.300]), manually calculate the probability of the sample being a good melon using the provided training data.

Optional: 计算'敲声'属性的条件概率时引入一个权重因子（例如乘以2），并说明这样做的原因是基于观察到的模型表现，以及这种改变可能对模型整体性能的影响.

In [3]:
```python
# %load Bayesian.py
import numpy as np
import math

#训练数据集:
dataSet = [
        ['青绿', '蜷缩', '浊响', '清晰', '凹陷', '硬滑', 0.697, 0.460, 1],
        ['乌黑', '蜷缩', '沉闷', '清晰', '凹陷', '硬滑', 0.774, 0.376, 1],
        ['乌黑', '蜷缩', '浊响', '清晰', '凹陷', '硬滑', 0.634, 0.264, 1],
        ['青绿', '蜷缩', '沉闷', '清晰', '凹陷', '硬滑', 0.608, 0.318, 1],
        ['浅白', '蜷缩', '浊响', '清晰', '凹陷', '硬滑', 0.556, 0.215, 1],
        ['青绿', '稍蜷', '浊响', '清晰', '稍凹', '软粘', 0.403, 0.237, 1],
        ['乌黑', '稍蜷', '浊响', '稍糊', '稍凹', '软粘', 0.481, 0.149, 1],
        ['乌黑', '稍蜷', '浊响', '清晰', '稍凹', '硬滑', 0.437, 0.211, 1],
        ['乌黑', '稍蜷', '沉闷', '稍糊', '稍凹', '硬滑', 0.666, 0.091, 0],
        ['青绿', '硬挺', '清脆', '清晰', '平坦', '软粘', 0.243, 0.267, 0],
        ['浅白', '硬挺', '清脆', '模糊', '平坦', '硬滑', 0.245, 0.057, 0],
        ['浅白', '蜷缩', '浊响', '模糊', '平坦', '软粘', 0.343, 0.099, 0],
        ['青绿', '稍蜷', '浊响', '稍糊', '凹陷', '硬滑', 0.639, 0.161, 0],
        ['浅白', '稍蜷', '沉闷', '稍糊', '凹陷', '硬滑', 0.657, 0.198, 0],
        ['乌黑', '稍蜷', '浊响', '清晰', '稍凹', '软粘', 0.360, 0.370, 0],
        ['浅白', '蜷缩', '浊响', '模糊', '平坦', '硬滑', 0.593, 0.042, 0],
        ['青绿', '蜷缩', '沉闷', '稍糊', '稍凹', '硬滑', 0.719, 0.103, 0]
    ]

feature = ['色泽','根蒂','敲声','纹理','脐部','触感','密度','含糖率']

#离散属性取值数目
Num = [3,3,3,3,3,2]
```

```
#测试样本1
testData = ['青绿','蜷缩','浊响','清晰','凹陷','硬滑',0.697,0.460]

def Bayesian(trainSet,testData):
    m = len(trainSet)                    #样本数
    goodMelon = []; badMelon = []
    for i in range(m):
        if trainSet[i][-1] == 1:
            goodMelon.append(trainSet[i])
        else:
            badMelon.append(trainSet[i])

    p_good = 1.0
    p_bad = 1.0

    #计算先验概率
    p_good = p_good*(len(goodMelon)+1)/(m+2)
    p_bad = p_bad*(len(badMelon)+1)/(m+2)

    #计算条件概率-离散
    d1 = len(Num)
    print("########好瓜属性条件概率-离散########")
    for i in range(d1):
        Dc = 0 #不同属性与样本属性相同的个数
        Ni = Num[i]
        for j in range(len(goodMelon)):
            if testData[i] == goodMelon[j][i]:
                Dc = Dc + 1
        p_g = (Dc+1)/(len(goodMelon)+Ni)
        print(testData[i],p_g)
        p_good = p_good*p_g

    print("########坏瓜属性条件概率-离散########")
    for i in range(d1):
        Dc = 0 #不同属性与样本属性相同的个数
        Ni = Num[i]
        for j in range(len(badMelon)):
            if testData[i] == badMelon[j][i]:
                Dc = Dc + 1
        p_b = (Dc+1)/(len(badMelon)+Ni)
        print(testData[i],p_b)
        p_bad = p_bad*p_b

    #计算条件概率-连续
    density_sugar_good = [sample[6:8] for sample in goodMelon] #提取连续值
    density_sugar_bad = [sample[6:8] for sample in badMelon] #提取连续值
    ave_good = np.mean(density_sugar_good, axis = 0)
    ave_bad = np.mean(density_sugar_bad, axis = 0)
    std_good = np.std(density_sugar_good, axis = 0, ddof=1) #ddof=1:样本标
    std_bad = np.std(density_sugar_bad, axis = 0, ddof=1)
    print("########好瓜属性条件概率-连续########")
    for i in range(2):
        p_g = 1/(np.sqrt(2*math.pi)*std_good[i])*math.exp(-(testData[6+i]
        print(feature[6+i],p_g)
        p_good = p_good*p_g
    print("########坏瓜属性条件概率-连续########")
    for i in range(2):
        p_b = 1/(np.sqrt(2*math.pi)*std_good[i])*math.exp(-(testData[6+i]
        print(feature[6+i],p_b)
        p_bad = p_bad*p_b
```

```
    #比较好瓜与坏瓜的概率
    if p_good>p_bad:
        print('预测结果为好瓜')
    else:
        print('预测结果为坏瓜')


Bayesian(dataSet,testData)
```

```
########好瓜属性条件概率-离散########
青绿 0.36363636363636365
蜷缩 0.5454545454545454
浊响 0.6363636363636364
清晰 0.7272727272727273
凹陷 0.5454545454545454
硬滑 0.7
########坏瓜属性条件概率-离散########
青绿 0.3333333333333333
蜷缩 0.3333333333333333
浊响 0.4166666666666667
清晰 0.25
凹陷 0.25
硬滑 0.6363636363636364
########好瓜属性条件概率-连续########
密度 1.9590115494650384
含糖率 0.7880520952044112
########坏瓜属性条件概率-连续########
密度 1.813364315379696
含糖率 0.07072938250447534
预测结果为好瓜
```

Part 2

Experiment Description

1. Explore the application of the Naive Bayes classifier in text classification tasks. Specifically, we use the fetch_20newsgroups dataset, which consists of about 20,000 news articles from various newsgroups.
2. To simplify the experiment, we focus on two categories: comp.graphics (computer graphics) and sci.space (space science). This allows us to set the problem as a binary classification task, determining whether a given news article discusses computer graphics or space science.

Experiment Objectives

1. Understand the Basics of Text Classification.
2. Master the Application of Naive Bayes Algorithm, including "make_pipeline" and "MultinomialNB", etc.

In [4]:

```python
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

# 加载数据集
categories = ['comp.graphics', 'sci.space']
data = fetch_20newsgroups(subset='all', categories=categories, shuffle=Tr

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(data.data, data.targe

# 创建一个管道，包含TF-IDF向量化和朴素贝叶斯分类器
model = make_pipeline(TfidfVectorizer(), MultinomialNB())

# 训练模型
model.fit(X_train, y_train)

# 预测测试集
predicted = model.predict(X_test)

# 输出模型的准确率和其他指标
print("Classification report for classifier %s:\n%s\n" % (model, classifi
print("Confusion Matrix:\n", confusion_matrix(y_test, predicted))
```

```
Classification report for classifier Pipeline(steps=[('tfidfvectorizer',
TfidfVectorizer()),
                ('multinomialnb', MultinomialNB())]):
              precision    recall  f1-score   support

           0       0.99      0.92      0.95       250
           1       0.93      0.99      0.96       240

    accuracy                           0.96       490
   macro avg       0.96      0.96      0.96       490
weighted avg       0.96      0.96      0.96       490


Confusion Matrix:
 [[231  19]
 [  3 237]]
```